UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026

Applied Parallel Programming

Lecture 23: Advanced Optimizations-Improving Attention

Hrishi Shah

04/16/2026

Course Logistics

Milestone 3 of CNN has been released, GPT will be released ~today.

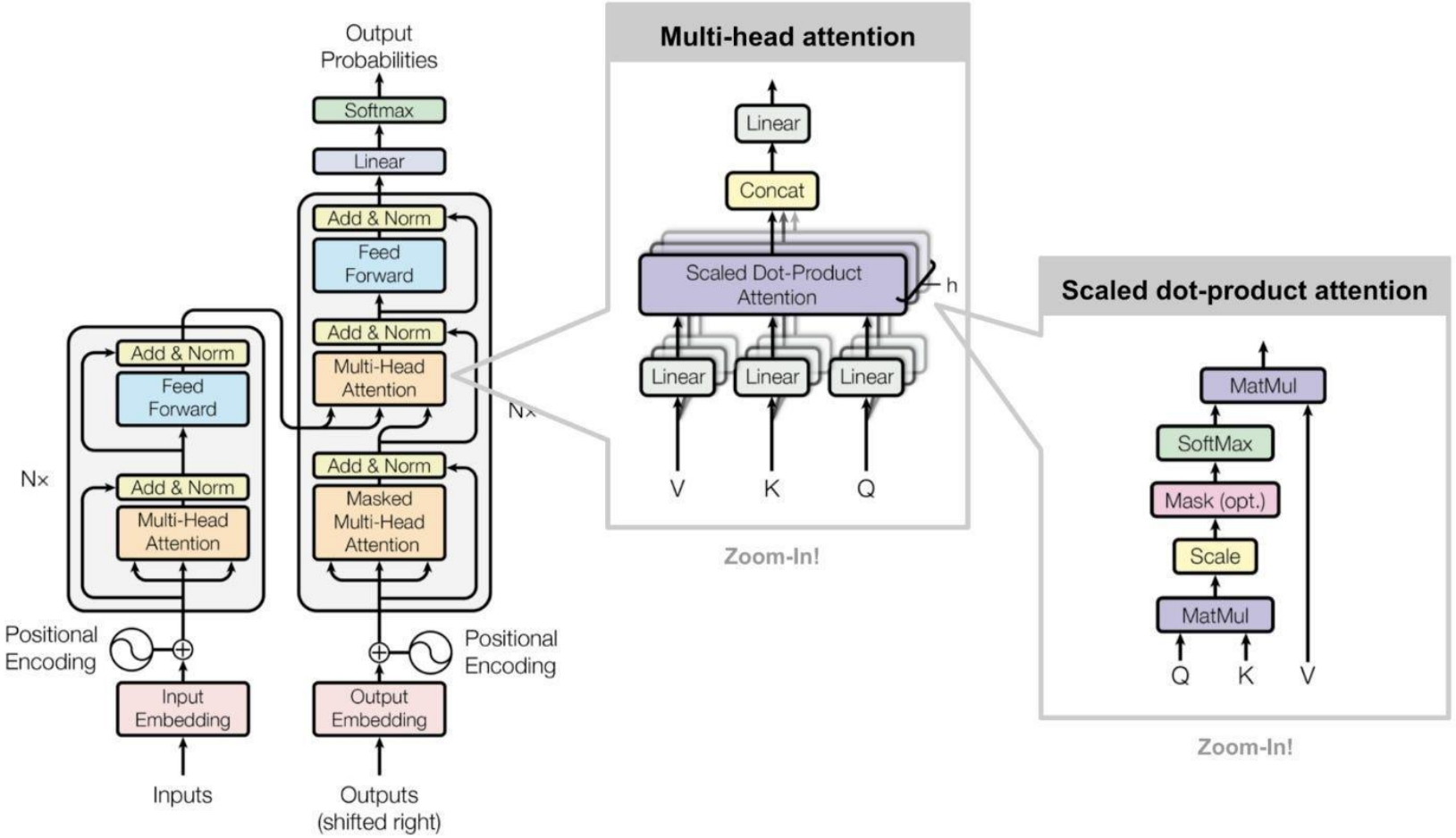
Exam 2 is approaching in a few weeks, study materials will be uploaded soon, can expect similar format and set of study resources as Exam 1.

There might be a shorter question about your project work that isn't specific to either of the two project tracks, can expect higher level questions and understanding checks.

Previous sems separate versions of Exam 2, may or may not be the same this semester (same time and place regardless).

Review: What is Attention?

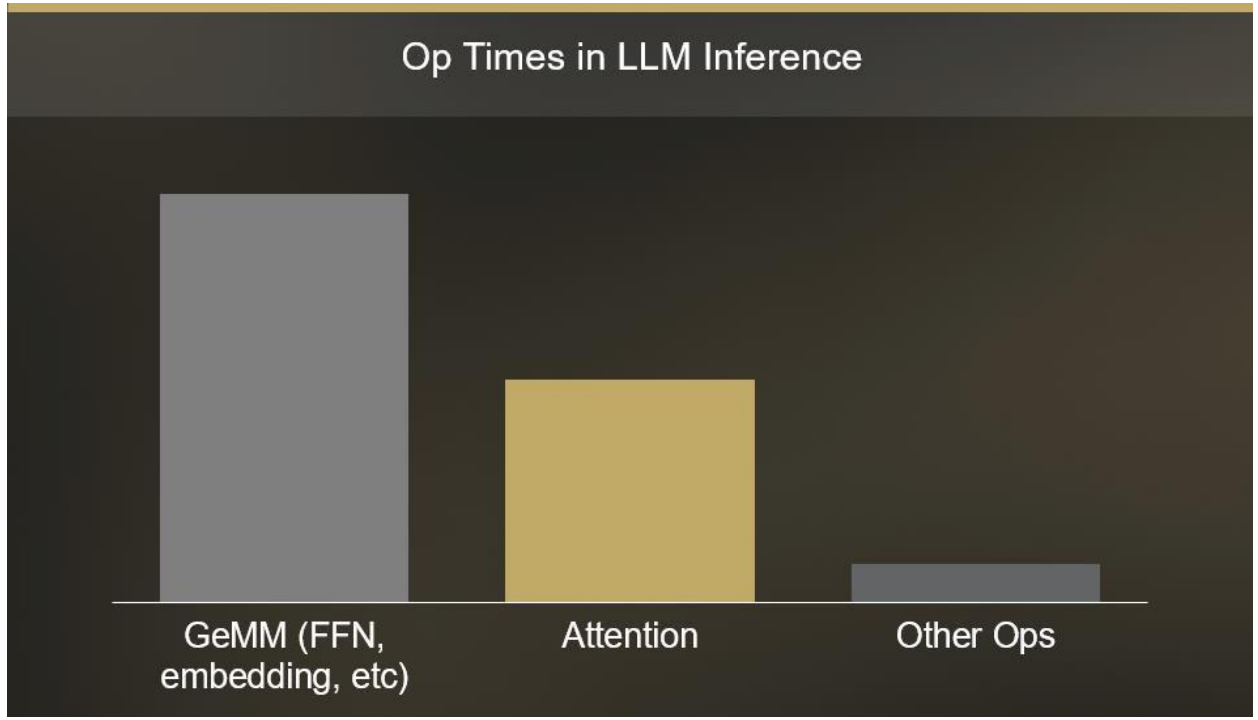
Main operation in modern LLMs that allows tokens to attend to each other, providing context across the text.



Why Optimize Attention

Attention is one of the major bottlenecks in LLM serving.

Significantly higher latency than all other operations (except for matmul) even in optimized LLM serving frameworks.



General Op Times Trend for LLM Inference

Basic Attention Design

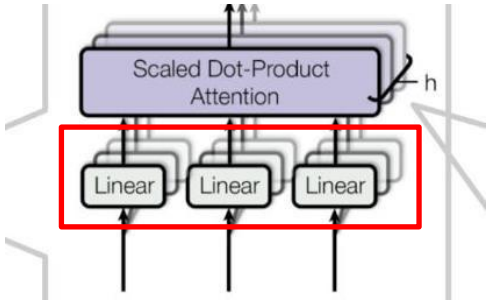
$$\mathbf{S} = \mathbf{Q}\mathbf{K}^{\mathrm{T}}$$

$$\mathbf{P} = \mathrm{softmax}(\mathbf{S})$$

$$\mathbf{O} = \mathbf{P}\mathbf{V}$$

Attention itself is just two matrix multiplications and a Softmax function.

However, to obtain the Q, K, and V vectors, we need to multiply each input token with a set of pretrained weights (and add biases) first.



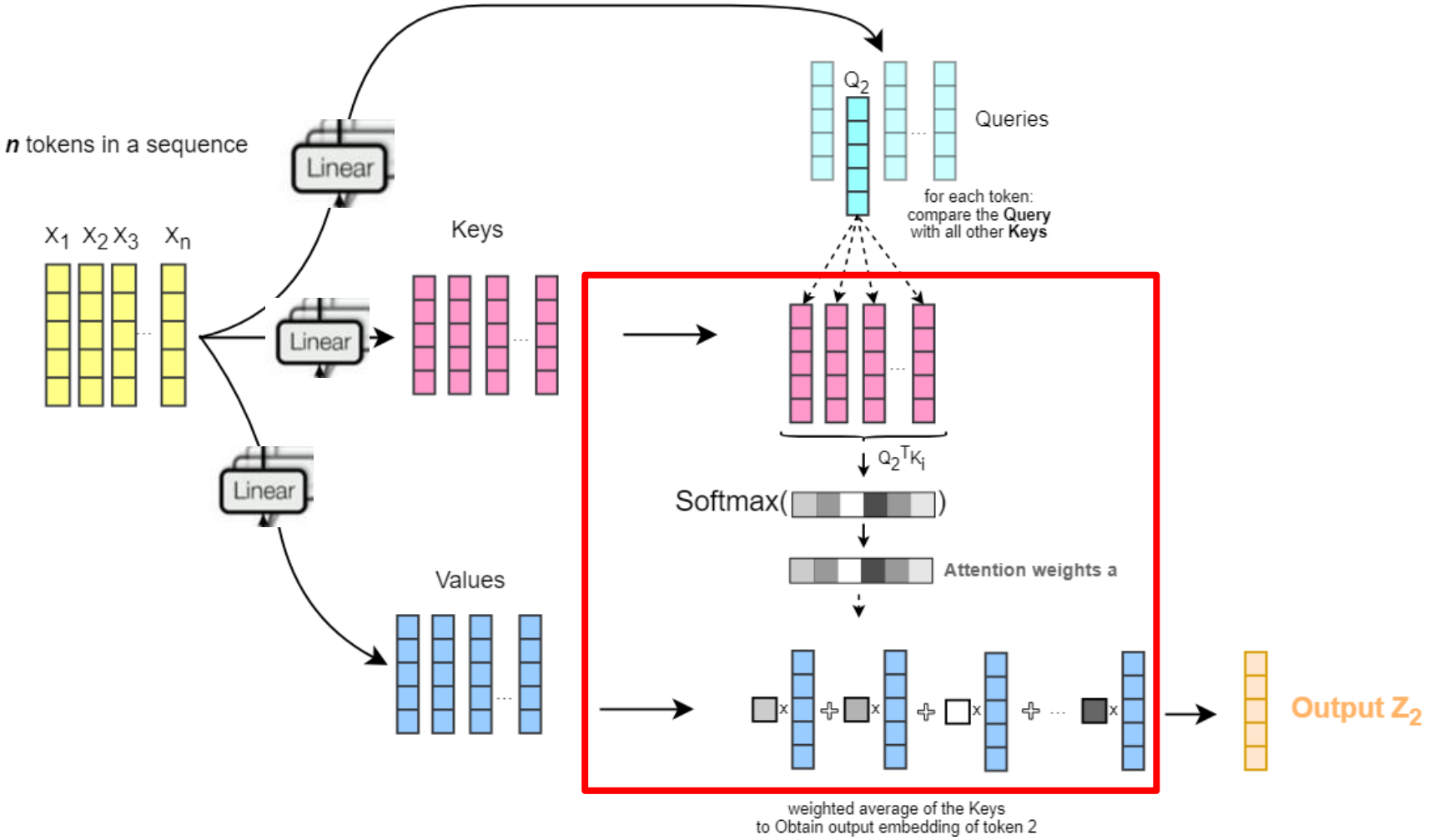
Q, K, and V projection (matmul)

Basic Attention Design

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^{\mathrm{T}}$$

$$\mathbf{P} = \mathrm{softmax}(\mathbf{S})$$

$$\mathbf{O} = \mathbf{P}\mathbf{V}$$



Implementation

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^{\mathrm{T}}$$
$$\mathbf{P} = \mathrm{softmax}(\mathbf{S})$$
$$\mathbf{O} = \mathbf{P}\mathbf{V}$$

Naively, attention can be implemented as three individual kernel calls:

- First kernel loads Q (**T**x**C**) and transposed K (**C**x**T**) from global memory, perform matmul, then store the result of size **T**x**T** back into global memory
- Second kernel loads the result (matrix of size **T**x**T**) from the previous kernel, performs Softmax on it, then stores the result (**T**x**T**) back into global memory
- Last kernel loads the **T**x**T** Softmax result and the V matrix from global memory, and performs the last PV matmul

Implementation

T is the sequence length of the inputs, or the max context length of the model…

For newer models, this parameter is anywhere in the tens of thousands, or even up to hundreds of thousands and millions.

Shoving a 100,000 x 100,000 matrix back and forth between global memory and on-chip memory is probably bad for performance.

There must be a smarter solution to these frequent memory transactions…

Kernel Fusion

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^{\mathrm{T}}$$

$$\mathbf{P} = \mathrm{softmax}(\mathbf{S})$$

$$\mathbf{O} = \mathbf{P}\mathbf{V}$$

To avoid moving the **T**x**T** matrix back and forth between on-chip and global memory, let's fuse all three kernels into one!

But not so fast…

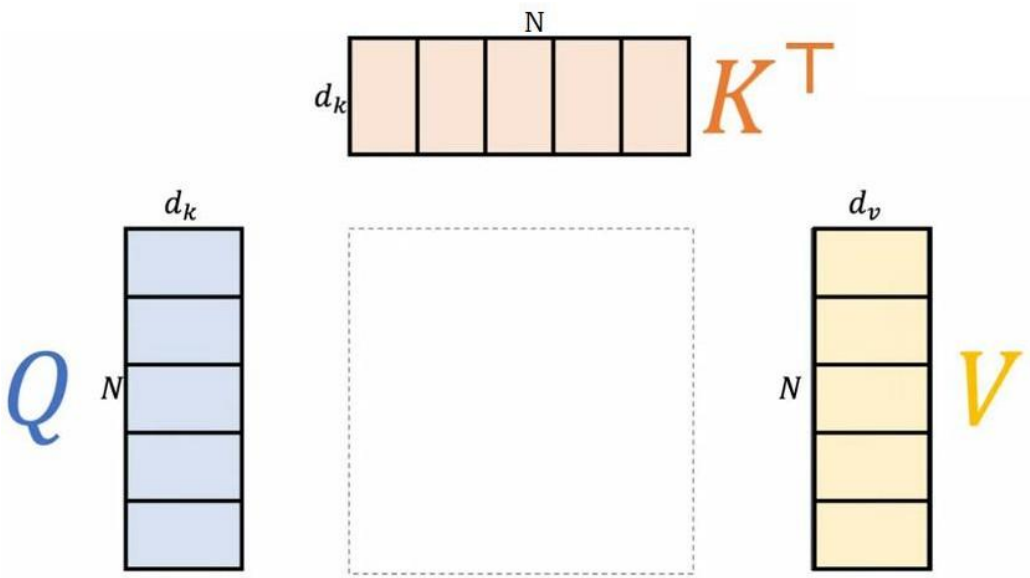
How big is shared memory again?

The A40 GPU has about 100KB per SM; for the B200 GPU, it's about 140KB

Good luck finding fast on-chip memory for your 100,000 x 100,000 matrix…

Kernel Fusion – Tiling?

What if we try to calculate small tiles of the **T**x**T** matrix, do Softmax and the PV matmul on that partial result, a little bit at a time, can we fuse that way?



Kernel Fusion – Tiling?

What if we try to calculate small tiles of the **T**x**T** matrix, do Softmax and the PV matmul on that partial result, a little bit at a time, can we fuse that way?

Too bad, Softmax requires the **max** and **sum** of each <u>full row</u> of the **T**x**T** matrix.

$$s\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

The Softmax Formula

You can't do softmax (or the PV matmul) until you have the full **T**x**T** matrix from the QK matmul.

And that's where people stopped… for some time…

Meanwhile…

The Online Softmax algorithm was developed and used for generating next tokens in language models that have extremely large vocabularies

(those models all predate the Transformer architecture).

It took ~4 years for people to apply this algorithm to transformers.

Online Softmax

Algorithm 2 Safe softmax

$m_0 \leftarrow -\infty$
for $k \leftarrow 1, V$ **do**
  $m_k \leftarrow \max(m_{k-1}, x_k)$
end for
$d_0 \leftarrow 0$
for $j \leftarrow 1, V$ **do**
  $d_j \leftarrow d_{j-1} + e^{x_j - m_V}$
end for
for $i \leftarrow 1, V$ **do**
  $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$
end for

The Softmax Algorithm

Algorithm 3 Safe softmax with online normalizer calculation

$m_0 \leftarrow -\infty$
$d_0 \leftarrow 0$
for $j \leftarrow 1, V$ **do**
  $m_j \leftarrow \max(m_{j-1}, x_j)$
  $d_j \leftarrow d_{j-1} \times e^{m_{j-1} - m_j} + e^{x_j - m_j}$
end for
for $i \leftarrow 1, V$ **do**
  $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$
end for

The Online Softmax Algorithm

Online Softmax algorithm keeps a running max value ***m*** and normalization term ***d*** as it iterates over the input, continuously rescaling earlier values when a new max appears. This eliminates the need to compute the global maximum upfront, allowing Softmax to be computed on an iterative basis.

Flash Attention

In May 2022, a research group at Stanford connected the dots, and the Online Softmax was incorporated in attention for transformers.

This led to **Flash Attention**, a monumental attention optimization that is now widely used in the industry!

arXiv > cs > arXiv:2205.14135

Search... All fields Search

Help | Advanced Search

Computer Science > Machine Learning

[Submitted on 27 May 2022 (v1), last revised 23 Jun 2022 (this version, v2)]

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré

Transformers are slow and memory-hungry on long sequences, since the time and memory complexity of self-attention are quadratic in sequence length. Approximate attention methods have attempted to address this problem by trading off model quality to reduce the compute complexity, but often do not achieve wall-clock speedup. We argue that a missing principle is making attention algorithms IO-aware -- accounting for reads and writes between levels of GPU memory. We propose FlashAttention, an IO-aware exact attention algorithm that uses tiling to reduce the number of memory reads/writes between GPU high bandwidth memory (HBM) and GPU on-chip SRAM. We analyze the IO complexity of FlashAttention, showing that it requires fewer HBM accesses than standard attention, and is optimal for a range of SRAM sizes. We also extend FlashAttention to block-sparse attention, yielding an approximate attention algorithm that is faster than any existing approximate attention method. FlashAttention trains Transformers faster than existing baselines: 15% end-to-end wall-clock speedup on BERT-large (seq. length 512) compared to the MLPerf 1.1 training speed record, 3× speedup on GPT-2 (seq. length 1K), and 2.4× speedup on long-range arena (seq. length 1K-4K). FlashAttention and block-sparse FlashAttention enable longer context in Transformers, yielding higher quality models (0.7 better perplexity on GPT-2 and 6.4 points of lift on long-document classification) and entirely new capabilities: the first Transformers to achieve better-than-chance performance on the Path-X challenge (seq. length 16K, 61.4% accuracy) and Path-256 (seq. length 64K, 63.1% accuracy).

Subjects: **Machine Learning (cs.LG)**
Cite as: arXiv:2205.14135 **[cs.LG]**
(or arXiv:2205.14135v2 **[cs.LG]** for this version)
https://doi.org/10.48550/arXiv.2205.14135

Submission history
From: Tri Dao [view email]
[v1] Fri, 27 May 2022 17:53:09 UTC (1,325 KB)
[v2] Thu, 23 Jun 2022 17:53:32 UTC (1,653 KB)

Access Paper:
View PDF
TeX Source
view license

Current browse context:
cs.LG
< prev | next >
new | recent | 2022-05
Change to browse by:
cs

References & Citations
NASA ADS
Google Scholar
Semantic Scholar

1 blog link (what is this?)

Export BibTeX Citation

Bookmark

Flash Attention

Using Online Softmax, Flash Attention fuses all three attention steps (QK matmul, Softmax, and the PV matmul) into one kernel.

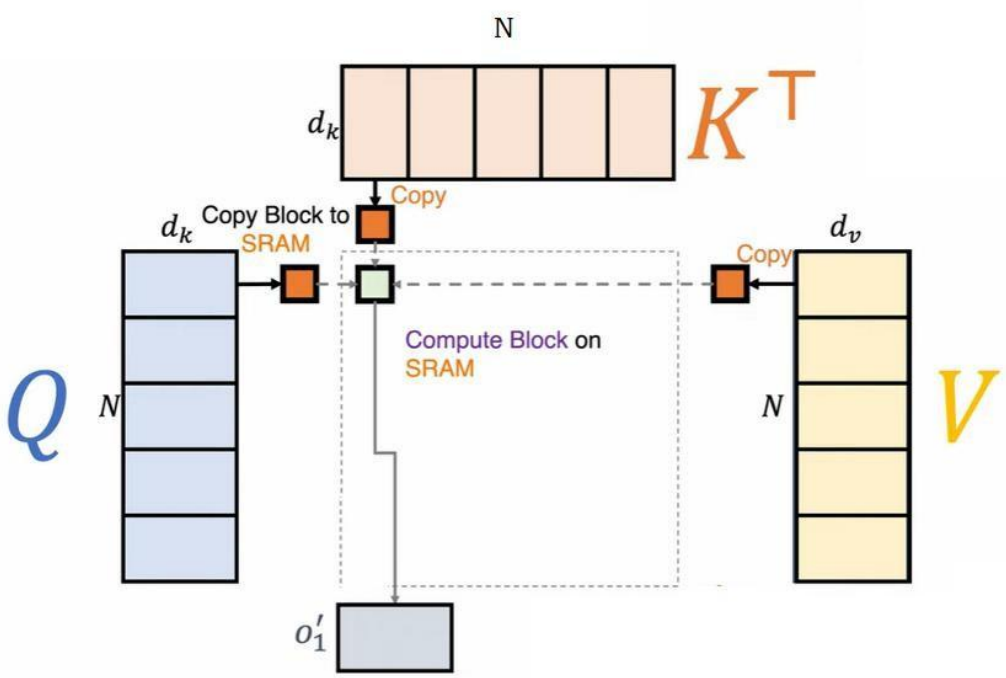
In a tiling fashion, Flash Attention loads tiles of Q, K, and V into fast on-chip memory (e.g. shared memory), and accumulates partial output values for a tile of Queries using each tile of Keys and Values.

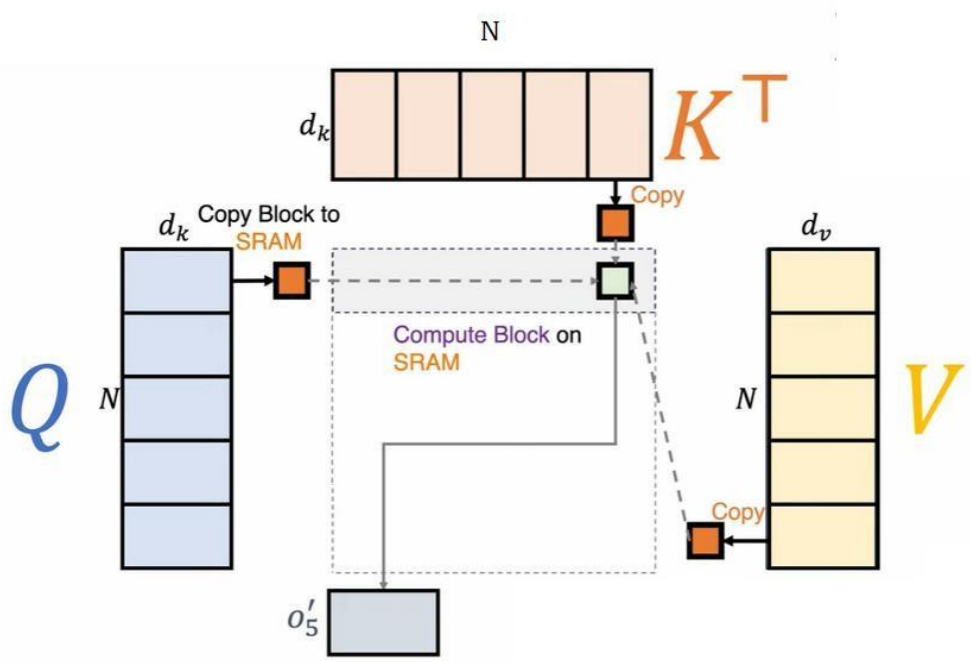
As of today, there have been 4 officially released versions of Flash Attention

- **FlashAttention**: introduced the idea of Tiled Attention using Online Softmax
- **FlashAttention2**: small algorithmic tweaks and discussions different work partitioning among warps
- *FlashAttention3*: focus on Hopper+ hardware (WGMMA, TMA, FP8, etc.)
- *FlashAttention4*: focus on Blackwell hardware (CTA scheduling)

Flash Attention Review

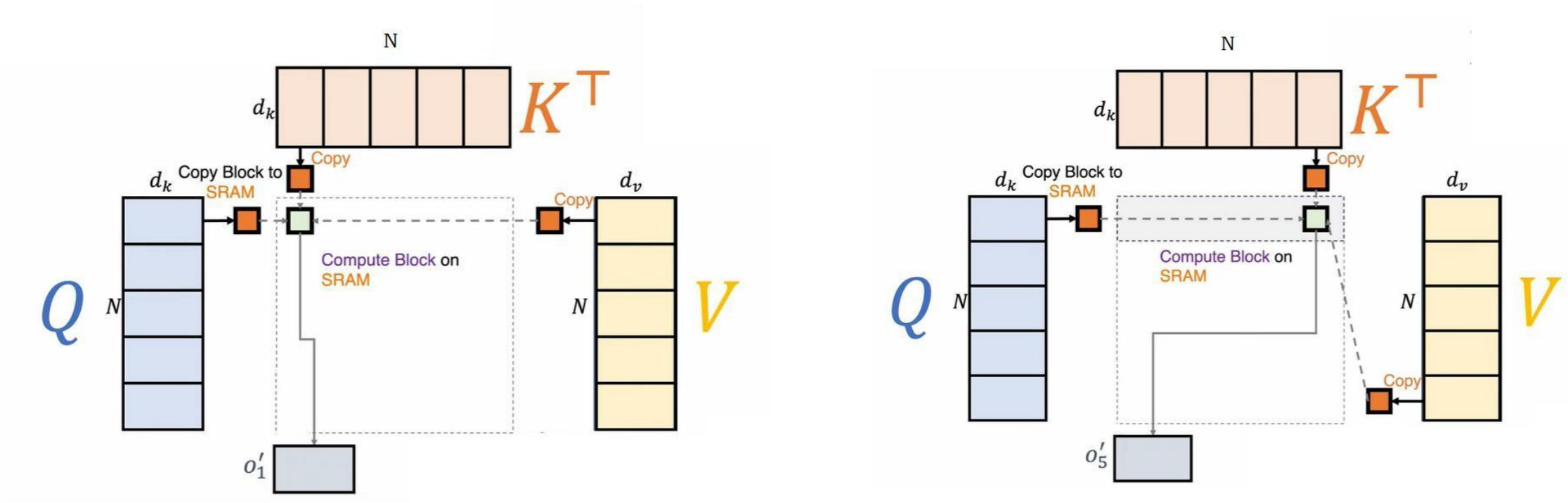
Flash Attention allows the QK (transposed) matmul, Softmax, and PV matmul to be fused into one kernel, executing in a tiling fashion similar to shared memory tiling or joint register and shared memory tiling for matrix multiplication.





[Helpful Video](#) on Flash Attention

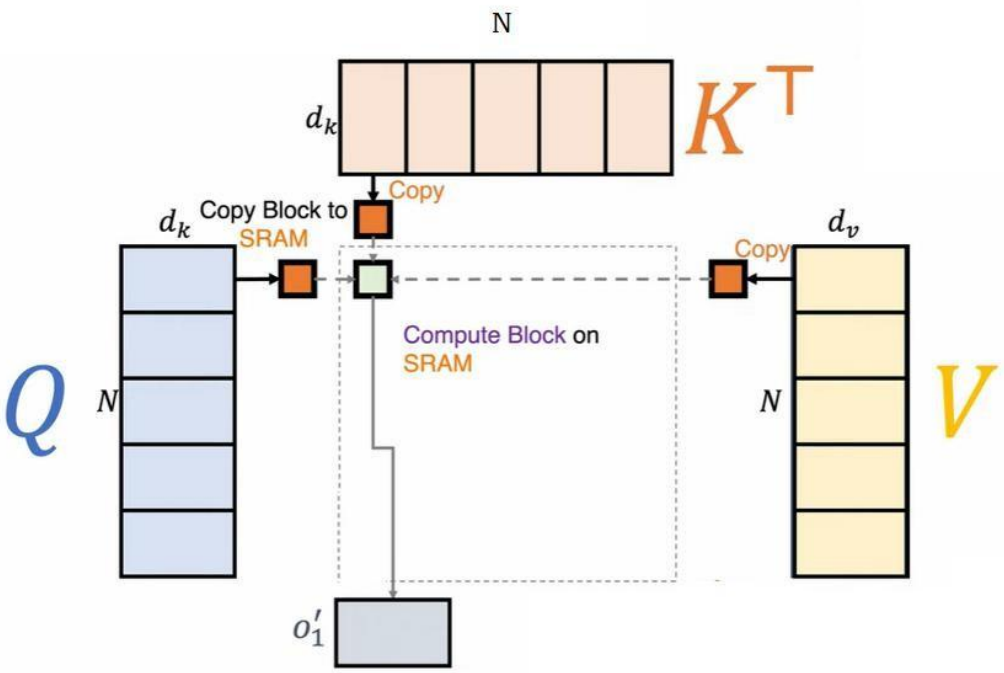
Flash Attention Review

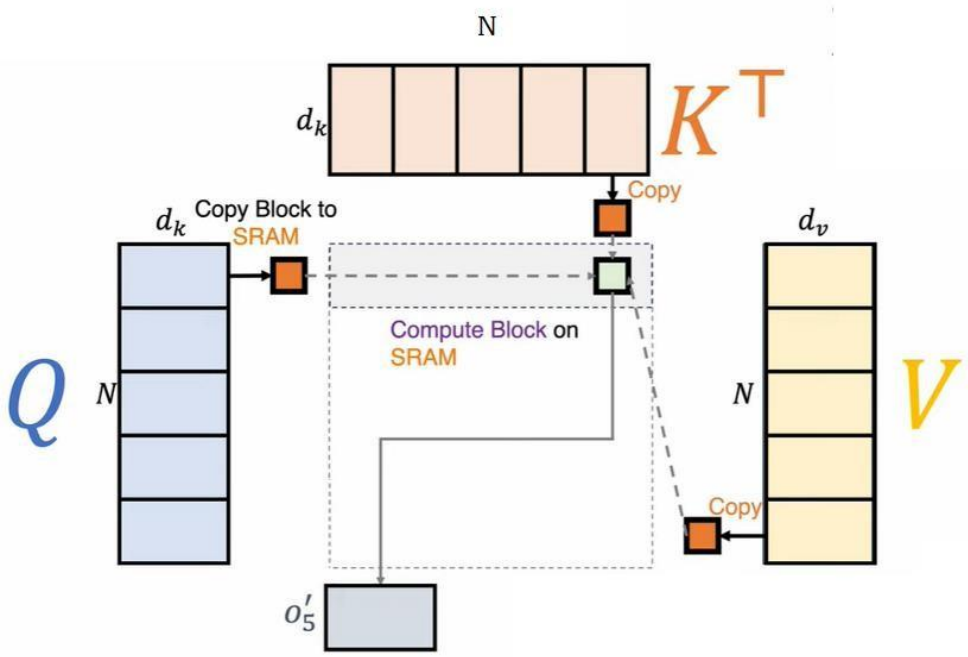


Helpful Video on Flash Attention

This optimization significantly reduces memory transfer overhead in the attention computation and can lead to 3x – 10x speedup depending on hardware and implementation (despite having the same FLOPs as basic attention).

Any Questions?





[Helpful Video](#) on Flash Attention

Full Sequence Attention is Costly

Even with FlashAttention, attention as an operation still scales in $O(T^2)$ time complexity, with T being the context/sequence length.

Many tasks also do not require long-range context, attending to tokens far back in time does not help in those cases (Translation, normal human conversation, etc).

In some use cases, the hardware available just isn't suited for computing long contexts efficiently (e.g. AI assistant on a smart watch).

In those cases, should we specifically design, train, and test new models for each use case?

You could, some do, but most of the time there are better ways.

Local/Windowed Attention

Reduces the attention context to a fixed local window of tokens.

Complexity drops from $O(\mathbf{T}^2)$ to $O(\mathbf{T} * \mathbf{W})$, where **T** is sequence length and **W** is window size.

If sequence is longer than window size **W**, just attend to the most recent **W** tokens and ignore all earlier tokens.

Easy to implement, quick to see performance improvements for most models.

But also loses long range context when you need it: chatbot might forget your name after some time, code generation becomes redundant as the code file becomes too large, etc.

Local/Windowed Attention

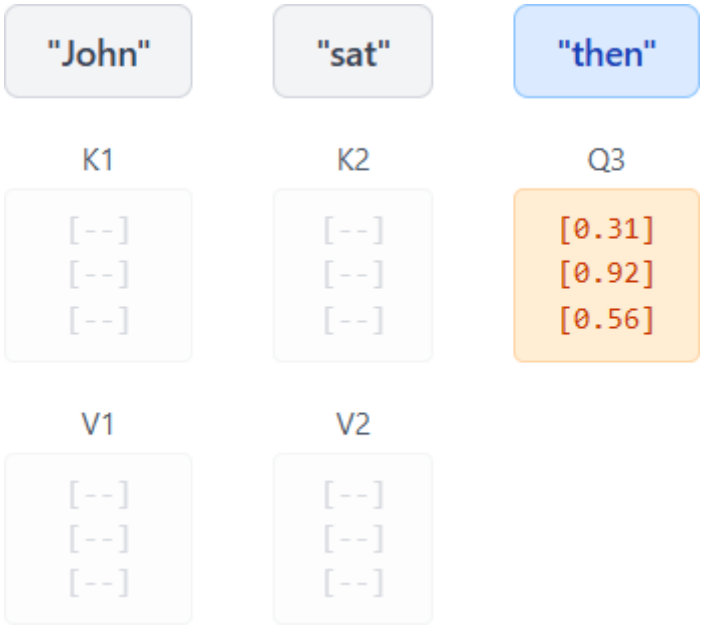
Can also mix and match:

- Periodically and/or algorithmically attend to full context to recover accuracy
- Dynamic window size for different types of prompts
- Strided windows, use increasingly long “jumps” to reach earliest tokens without $O(\mathbf{T}^2)$ complexity

Tricky to tune and hard to recover satisfactory amount of accuracy for all use cases, use with caution.

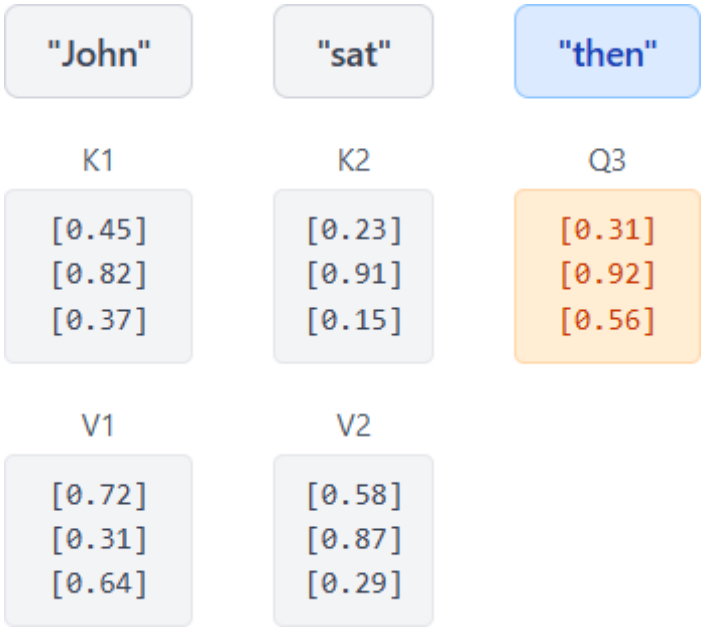
(In the GPT Project, you can just do it naively, but you’re encouraged to try out some of the strategies above in your own time).

Next Token Generation



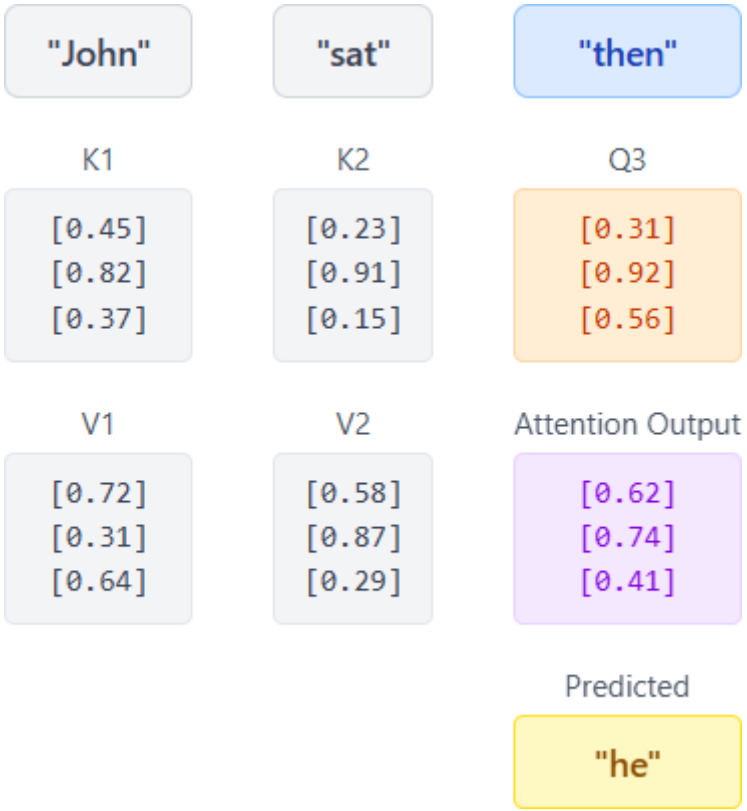
Given a prompt of 2 tokens, the Query token "then" computes attention over ***all*** previous tokens (context window) using matrix multiplication.

Next Token Generation



Given a prompt of 2 tokens, the Query token "then" computes attention over ***all*** previous tokens (context window) using matrix multiplication.

Next Token Generation

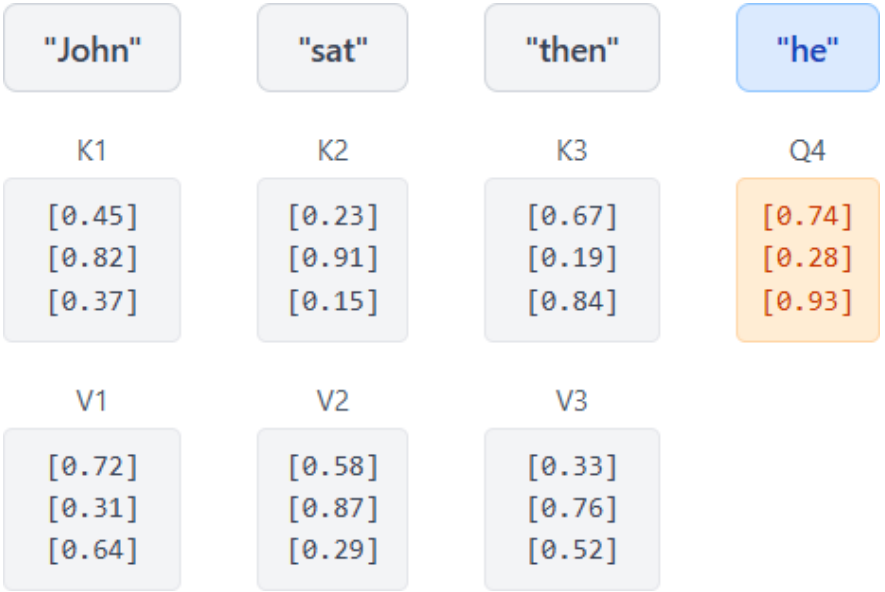


With the context accounted for, a new token can be generated (“he”).

Next Token Generation

Now with the new token generated, we append it to the context, and restart the whole process, to determine how much "*attention*" the new token should pay to all previous tokens.

Next Token Generation

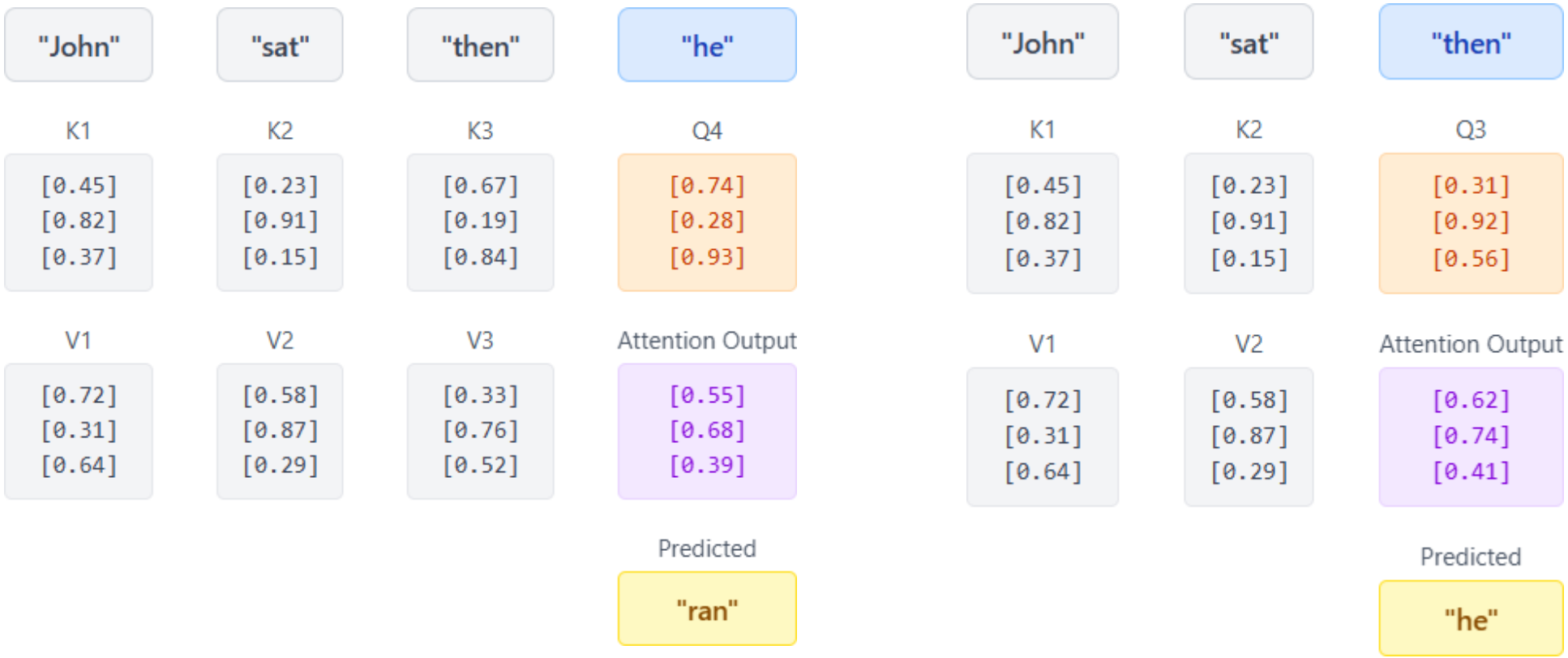


Now with the new token generated, we append it to the context, and restart the whole process, to determine how much "*attention*" the new token should pay to all previous tokens.

Next Token Generation

After recalculating all previous tokens' Q and K values, performing attention for the token "he", a new token can be generated ("ran").

Any Sharp Eyes?



There is some room for improvement here, anyone see it?

Redundant Key and Value Calculations

The Key and Value vectors of each token does not change when generating new tokens, yet we're always recalculating them.

Without any kind of reuse of these K and V vectors, at the 50th token in next token generation, there needs to be 50 Key and Value computations; At the 500th token, there needs to be 500 computations.

The total operations grow quadratically.

For a sequence of 1000 tokens, you end up doing about 500,000 redundant calculations.

This is not the kind of compute-bound you'd want…

KV Cache

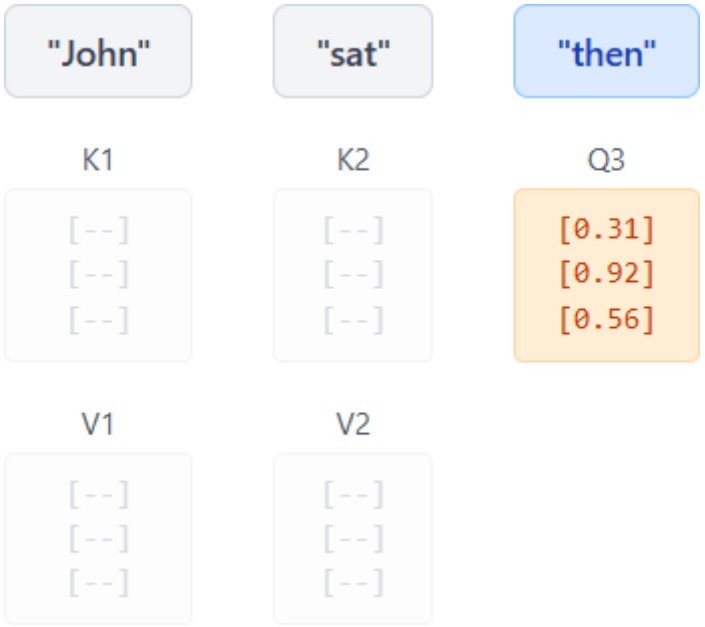
A KV Cache is designed to save and reuse previously generated Key and Value vectors in inference.

With the use of a KV cache, per-token generation cost is reduced roughly from $O(n^2)$ to $O(n)$ work.

This means attention as a mostly compute-bounded operation will now shift to being memory-bound, since most computations will be replaced by memory reads.

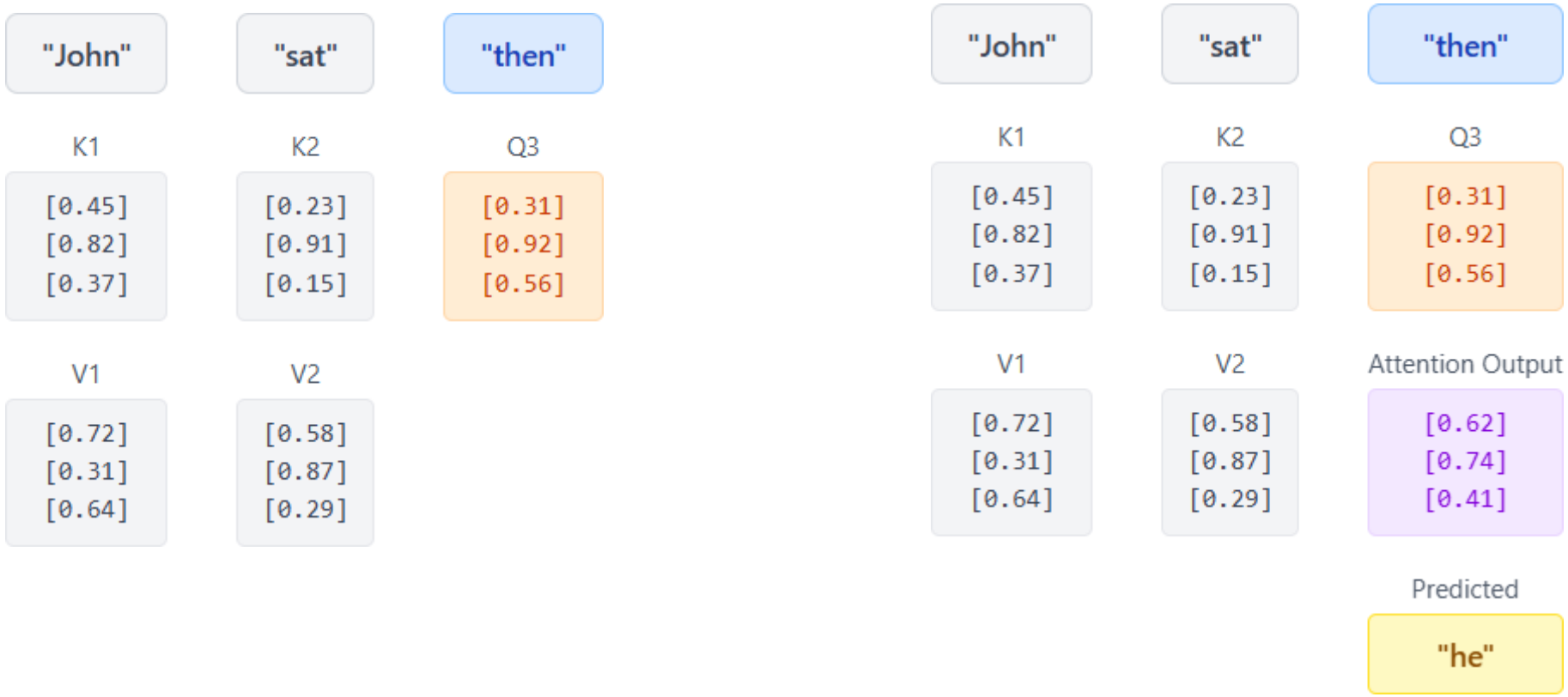
So how does a KV Cache work?

Let’s See the KV Cache In Action



Given a prompt of 2 tokens, the Query token "then" computes attention over ***all*** previous tokens (context window) using matrix multiplication.

Next Token Generation – KV Cache

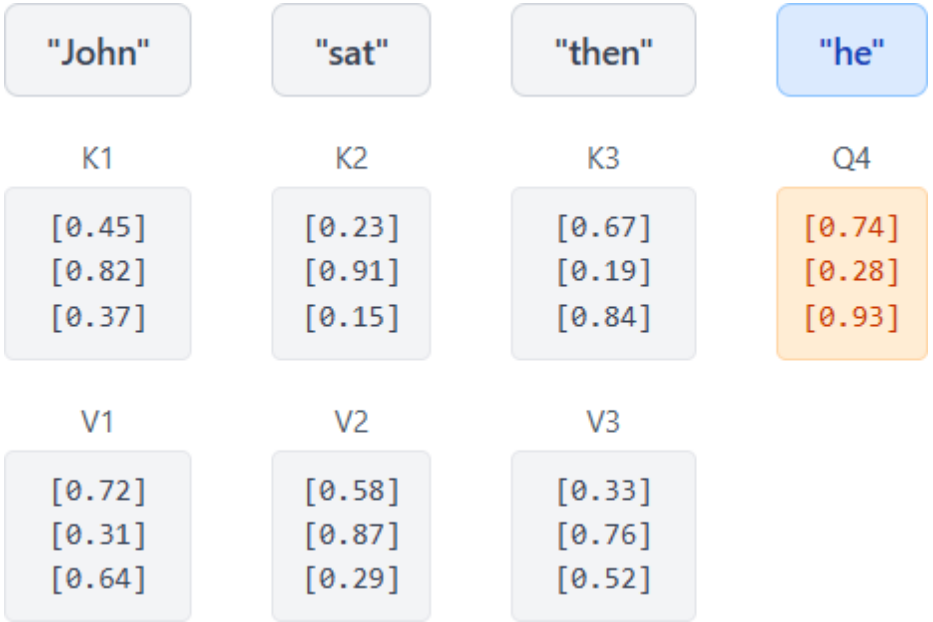


In the prefill stage, all prompt tokens are attended, a new token is generated (“he”)

Next Token Generation – KV Cache

However, with the KV cache, the Key and Value vectors for the previous tokens are saved and can be reused, but we still need to compute the K and V of the **most recently generated token** (“then”).

Next Token Generation – KV Cache



However, with the KV cache, the Key and Value vectors for the previous tokens are saved and can be reused, but we still need to compute the K and V of the **most recently generated token** (“then”).

Next Token Generation – KV Cache

Now, a new token “ran” is generated, without recomputing ***all*** previous Key and Value vectors (except for the most recently generated token).

KV Cache

Despite its name, most KV cache designs do not actually store the previous Key and Value vectors in any on-chip cache structure.

Global memory is usually the only choice, since Shared Memory or Registers are way too small on most modern GPUs for this purpose.

To implement a KV cache, there are many moving parts:

- Generating next tokens will now require keeping track of persistent states between forward pass calls across different batches
- Many existing kernels require redesign
- Position embedding logic must remain consistent between prefill and decode
- Potentially large KV cache memory needs to be properly managed

What Happens When KV Cache is Huge?

In the real world/industry scale of inference serving (like chatgpt.com), you often need to process thousands of different sequences from different users.

This means your implementation need to be capable of handling large number of batches efficiently. At this scale, small inefficiencies matter a lot more than you think.

For example, if you have an ***A100 GPU***, which has about ***80GB*** of global memory, a typical ***70B*** llama2 model will need about ***32KB*** of KV cache space per token per layer, so to respond to a single user prompt it's about ***10GB*** of memory usage just from the KV cache alone.

Memory Fragmentation

Moreover, if the KV cache memory is managed naively, memory fragmentation can become a serious issue (both internal and external fragmentation).

This can further limit the capabilities of your overall system performance.

(Those of you that have taken an OS design course should be familiar with this)

But speaking of OS design, if you took *ECE 391*, you should also know that there's a well-known solution in Operating Systems for issues like this:

Paging!

Memory Fragmentation in LLM Serving

In LLM serving, batching requests is mandatory to maintain GPU utilization.

Challenge: Cannot predict how many tokens will be generated, pre-allocate massive chunks according to request's maximum possible sequence length.

Problems:

- Internal fragmentation – If a model stops generating after a few tokens, the rest of the allocation is wasted
- External fragmentation – Allocation size is a factor of input sequence length and # of gen tokens, leaves free space between chunks
- Reservation waste – Constant size throughout generation, locking large chunks for entire request

This is What Happens When OS Designers Learn About LLMs

Inspired by the Virtual Memory Paging for Operating Systems, a group of researchers proposed PagedAttention in 2023.

The main idea of PagedAttention is to use fixed-size pages (blocks) and page tables to decouple logical KV cache positions from physical memory locations.

Each request's KV cache is broken into smaller pages, and each page contains Key Value vectors for a fixed number of tokens.

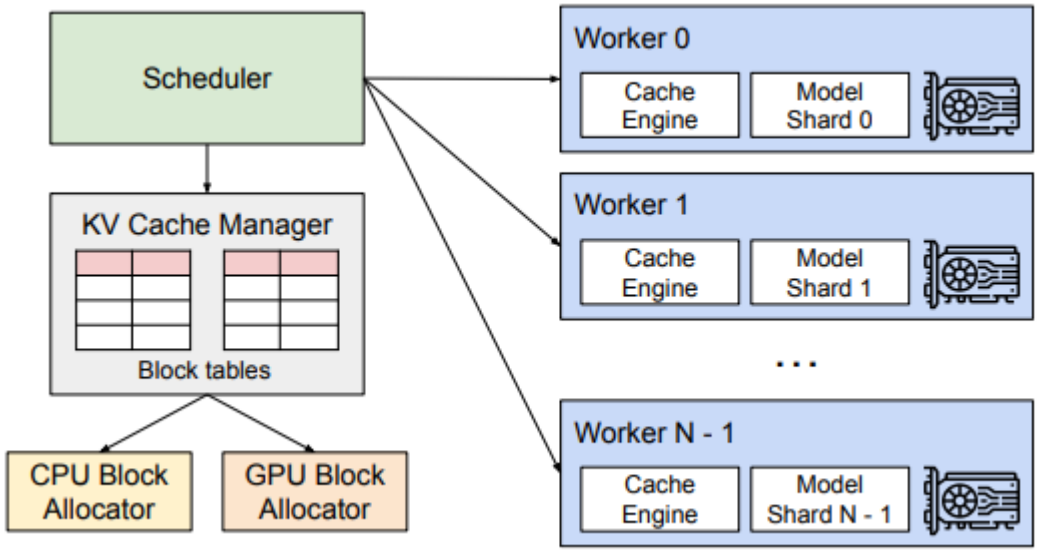
This design allows for much more flexibility in KV Cache memory management, as new physical blocks can be allocated when new tokens are being generated.

Block Allocator for KV Cache

Instead of forcing the cache to be a contiguous chunk, divides the cache into blocks (i.e. 16 tokens per block).

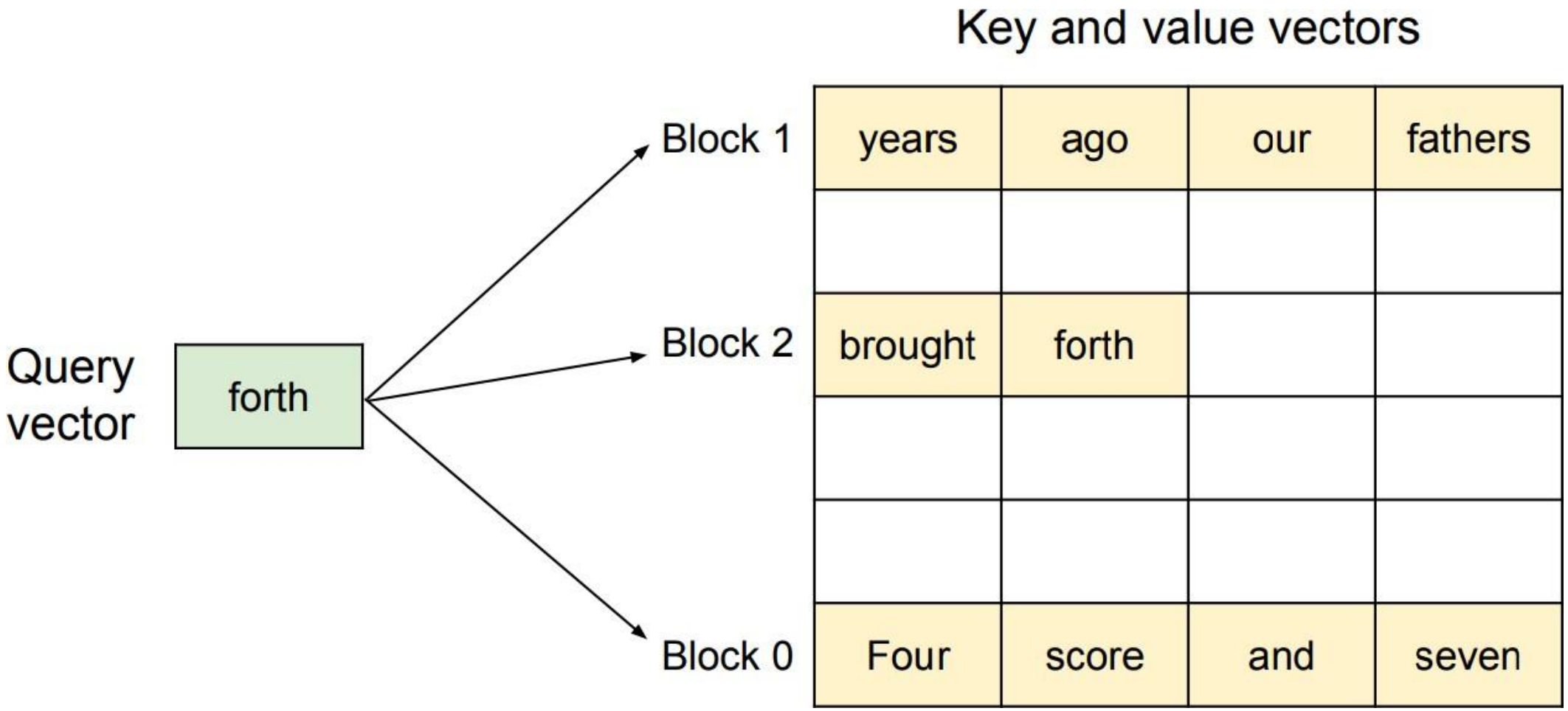
Model views the cache as a contiguous logical space.

When a physical block fills up, next block gets allocated.



When the GPU allocator runs out of space, CPU receives entire KV cache until other requests are completed and the swapped request can resume.

Paged Attention



What If We Want KV Cache to be Even Smaller?

Sometimes the raw size of large language models is simply too much for your hardware.

Imaging paying $30,000 for a single NVIDIA H100 GPU for your AI startup.

The KV cache for a single 4k token user session takes 4GB on your GPU for your baseline 7B model, and the model parameters takes up almost 30GB of DRAM
The GPU has only 40GB of DRAM…

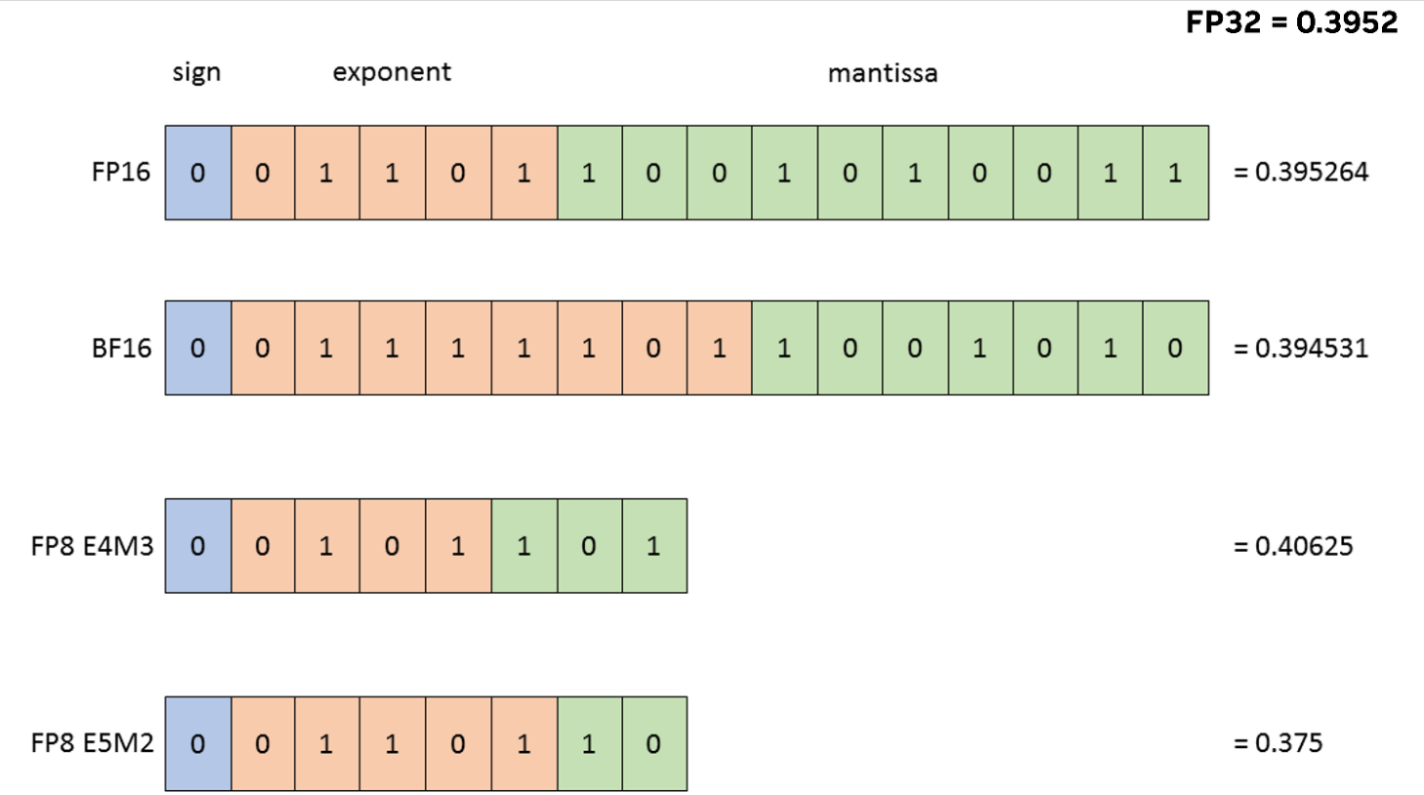
Good luck telling your investors that you can only serve 2 users at full context length before running out of memory.

Quantization

Ever thought about taking away some bits off of the Key and Value vectors?

Maybe the model won't notice and we'll be fine…

This is called quantization.



Quantization

Ever thought about taking away some bits off of the Key and Value vectors? Maybe the model won't notice and we'll be fine…

A model generated a 32-bit value to be stored in the KV Cache, but let's just chop off some mantissa and exponent bits to make it 16 bits so we can double our KV Cache size!

What if I want to quadruple it? I'll just chop off more bits to make all KV cache values 8-bit floating point!

4x KV Cache size!

Quantization

The shareholders are going to love this.

But this sounds like cheating, and cheating has consequences…

When you quantize from 32 bits to 8 bits, a lot of precision is lost, and to avoid losing too much language modeling accuracy, you'll need to implement sophisticated algorithms to handle things like overflow and scaling factors.

And wait, how do you even know how accurate a quantized model is?

If say you quantize to FP16, and your next token generation output seems like coherent English, does that mean you got a 2x speedup for free?

Model Evaluation – The Perplexity Scoring System

In order to evaluate your model's language modeling capabilities, you need to do more than a simple comparison of raw output values (or eyeballing generated text).

Introducing: the **Perplexity (PPL) Scoring System**

PPL is the most common metric for evaluating autoregressive language models (e.g. GPT family of models).

Perplexity measures how well a model predicts the next token, and is mathematically defined as the exponentiated average negative log-likelihood:

$$\mathrm{PPL}(X) = \exp\left\{-\frac{1}{t}\sum_{i}^{t}\log p_\theta(x_i|x_{<i})\right\}$$

Don't worry, we won't ask this formula in an exam

Model Evaluation – The Perplexity Scoring System

The PPL score is based on the model's predicted probability for every next token, the algorithm used to convert words into tokens, and a test dataset.

The output is a single number: how "surprised" the model is on average.

A low Perplexity score means the model predicts the data well.

A high Perplexity score means the model consistently assigns low probability to the true tokens.

Note that the dataset choice is very important, since perplexity is a measure of how well the model predicts ***that specific dataset's distribution***, and not an absolute measure of how good the model is.

Model Evaluation – The Perplexity Scoring System

There are many established datasets for the Perplexity system

Some notable examples are

- Wikitext-2
- C4
- gsm8k

Table 1: Comparison of Perplexity of Quantized LLaMa Models on C4 and WikiText-2 Datasets.

Method	Avg bit	C4 ↓	WikiText-2 ↓
LLaMa-7B	16	5.22	5.68
GPTQ [6]	4.0	5.62	8.14
OWQ [10]	4.01	5.56	7.15
LLM-QAT [12]	4.0	7.40	10.90
PB-LLM-20% [16]	3.4	20.61	17.19
APTQ	**4.0**	**5.23**	**6.45**
APTQ-75%	3.5	5.54	6.54
APTQ-50%	3.0	6.24	6.76

Example Source

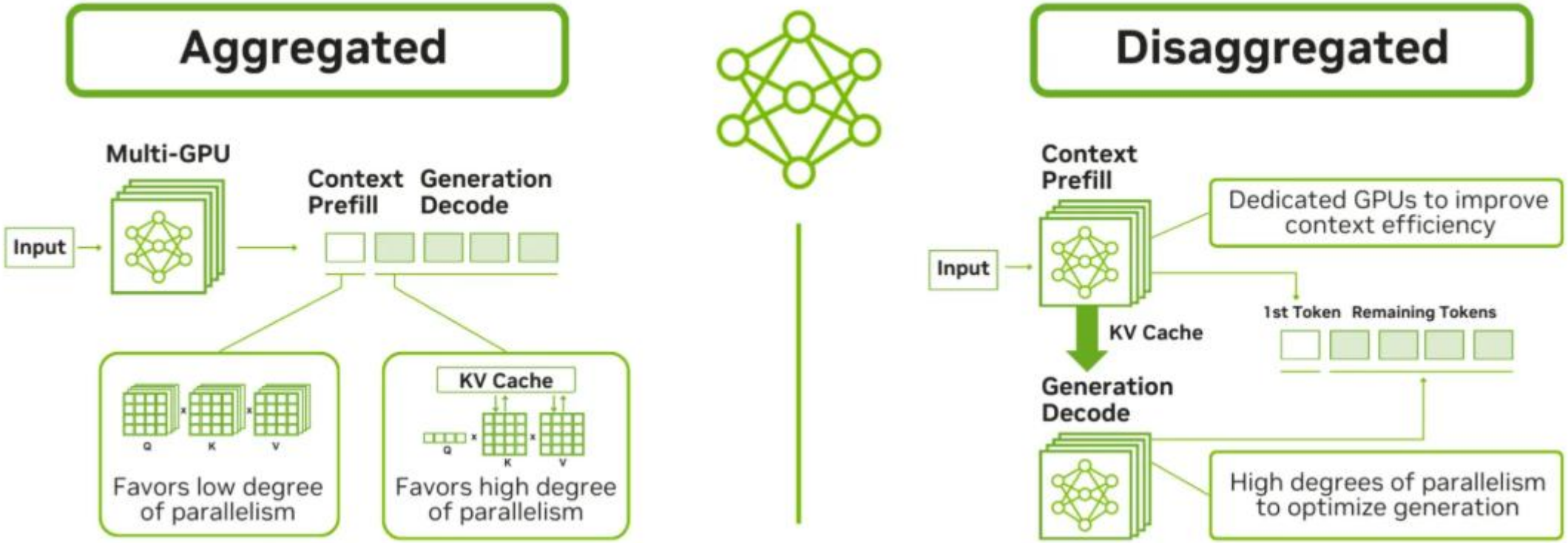
Two Phases of LLM Inference

Phase 1: Prefill (Prompt Processing)

- Processes the entire user prompt in parallel to compute the KV (Key-Value) cache.
- Profile: Compute-bound. Highly parallel, relies heavily on matrix multiplications (FLOPs).

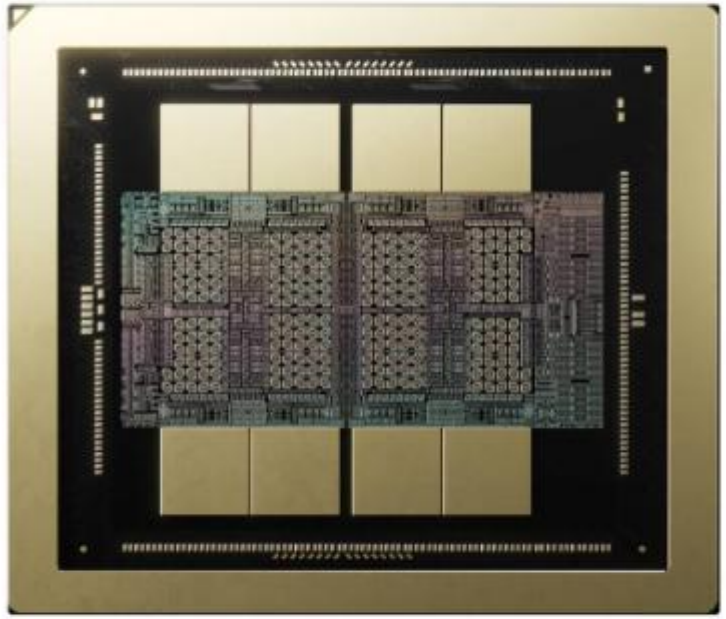
Phase 2: Decode (Token Generation)

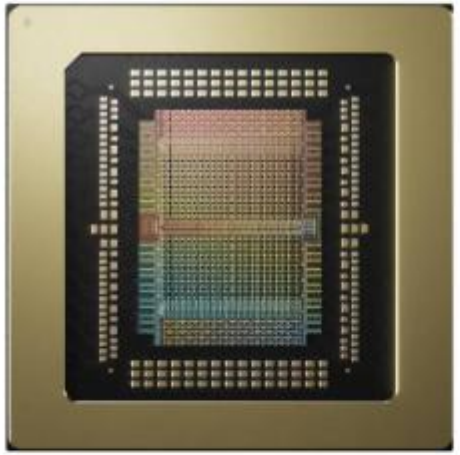
- Generates the response autoregressively, one single token at a time.
- Profile: Memory-bandwidth bound. Requires constantly reading the KV cache for every single token generated.


Aggregated
Multi-GPU
Input
Context Prefill
Generation Decode
Q
K
V
Favors low degree of parallelism
KV Cache
Q
K
V
Favors high degree of parallelism
Disaggregated
Context Prefill
Input
Dedicated GPUs to improve context efficiency
1st Token
Remaining Tokens
KV Cache
Generation Decode
High degrees of parallelism to optimize generation

Uniting Processors of Extreme FLOPS and Bandwidth

NVIDIA Rubin GPU + Groq 3 LPU





288 GB HBM4	500 MB SRAM
22 TB/s	**150 TB/s** SRAM Bandwidth
50 PFLOPs (NVFP4)	1.2 PFLOPs (FP8)

Questions?